Community Microgrid Energy Manager: Brief for the AI/ML Lead

Oct 2, 2026 · @Res

The big picture

You are building the brain of a small community power network. Every time step it reads grid status, solar output, household demand and battery charge, then decides where each unit of energy goes: into households, into the battery, or to and from the grid. The goals are lowest community cost and **critical loads never going dark**.

The physical setup (PS #38, Community Microgrid & Energy Sharing) is three neighbouring households on one shared 12 V DC bus:

- **Houses 1 and 2** have rooftop solar; House 3 has none. A house with surplus solar sells it to the others.
- **One community battery** (BESS, battery energy storage system) is shared by all.
- **A utility grid connection** can fail. When it does, the community runs alone ("islanded") on solar and battery.
- **Each house has a prepaid RFID meter.** Energy bought and sold moves credit between household wallets.

The idea is the same as commercial BESS analytics platforms (like AURA, see the last section): forecast, then **arbitrage**. Store energy when it is cheap or plentiful (midday sun, off-peak grid) and use it when it is expensive or scarce (evening peak, grid outage).

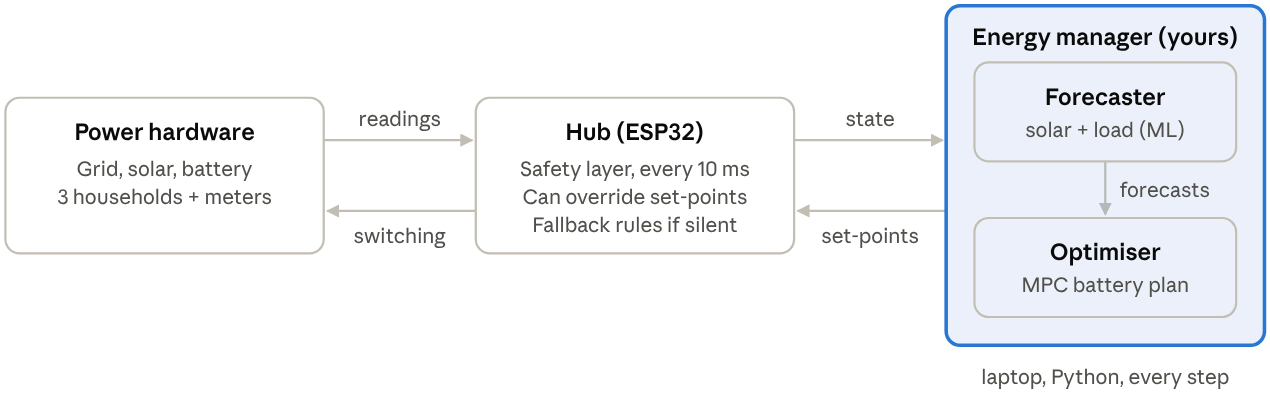
There are two layers. You own the top one; the hardware team owns the bottom one.

Layer	Runs on	Time scale	Decides
Energy manager (yours)	Laptop, Python	Every 1 min (demo) / 15 min (simulated day)	Battery charge/discharge power, grid import/export, solar allocation between houses, which non-critical loads to shed, trade prices

Layer	Runs on	Time scale	Decides
Safety layer (hardware hub, ESP32)	Microcontroller	Every 10 ms	Executes your set-points, but overrides them on any fault: grid loss, over-current, battery limits, low credit

The hub always has the last word. If your manager crashes or sends something unsafe, the hub falls back to simple built-in rules, so you can experiment freely.

Your manager decides; the hub executes and can override



control loop · state goes up over MQTT, set-points come back

Each step: the hub reports state, your forecaster predicts the next 24 h, the optimiser plans the battery, and the hub carries out the first step of that plan.

Power terms in plain language

You need about ten terms. The one that trips most people is **power vs energy**.

Term	Meaning	Think of it as
Power (W, kW)	Rate of energy flow right now	Speed of water through a pipe
Energy (Wh, kWh)	Power × time; what you pay for (1 kWh = 1 "unit" on an Indian bill)	Litres in the tank
BESS	Battery energy storage system: the community battery	A water tank that can fill and drain

Term	Meaning	Think of it as
SOC	State of charge: battery fill level, 0–100 %	Tank level
Capacity	Energy the battery can hold (e.g. 100 Wh in our demo)	Tank size
C-rate / power limit	Max charge or discharge power (e.g. 0.5C = half the capacity per hour)	Pipe width
Round-trip efficiency	Energy out ÷ energy in; about 90 % for LiFePO4	A 10 % "tax" on storing energy
Islanding	Disconnecting from a failed grid and running on local solar + battery	Switching to the backup tank
ToD tariff	Time-of-day price: cheap off-peak, expensive at the evening peak	Surge pricing
Arbitrage	Buy (charge) when cheap, sell or use (discharge) when expensive; profit = price gap minus losses	Buy low, sell high
Critical load	Must never lose power (medical device, lights)	Hard constraint, not an objective
Load shedding	Turning off low-priority loads when supply is short	Graceful degradation
Prosumer	A house that both produces (solar) and consumes	A node that is both buyer and seller

Inputs: what you get every time step

Each step the hub sends one JSON message with the current state. Power is in watts (the demo runs at tens of watts; a real community is the same maths in kW).

Field	Unit	Meaning
`t`	ISO time	Start of this step
`dt_min`	minutes	Step length (1 in the live demo, 15 in simulated days)

Field	Unit	Meaning
`grid_available`	bool	Is the utility grid up? Can change without warning
`price_buy`	₹/kWh	Cost of energy imported from the grid now (ToD tariff)
`price_sell`	₹/kWh	What the grid pays for exported energy (lower than buy)
`pv_w`	W per house	Solar output now, e.g. `{"H1": 14.2, "H2": 9.8, "H3": 0}`
`load_crit_w`	W per house	Critical demand (never shed)
`load_flex_w`	W per house	Non-critical demand (sheddable)
`soc`	0–1	Battery state of charge
`batt`	object	Fixed specs: `capacity_wh`, `p_charge_max_w`, `p_discharge_max_w`, `eff_roundtrip`, `soc_min`, `soc_max`
`credit`	₹ per house	Prepaid wallet balance from the RFID meters

Optional extras you can use if your method needs them:

- **Price schedule for the day**: known in advance (tariffs are published).
- **Planned outage hours**: Indian distribution companies often announce scheduled load-shedding, so some outages are predictable.
- **History** of every past message, for training forecasters.

```
{"t": "2026-10-02T18:15:00+05:30", "dt_min": 15, "grid_available": true,
 "price_buy": 9.5, "price_sell": 3.0,
 "pv_w": {"H1": 1.2, "H2": 0.8, "H3": 0},
 "load_crit_w": {"H1": 3, "H2": 3, "H3": 3},
 "load_flex_w": {"H1": 6, "H2": 4, "H3": 8},
 "soc": 0.72,
 "batt": {"capacity_wh": 100, "p_charge_max_w": 30, "p_discharge_max_w": 30,
          "eff_roundtrip": 0.9, "soc_min": 0.2, "soc_max": 0.95},
 "credit": {"H1": 42.0, "H2": 18.5, "H3": 2.1}}
```

The numbers in the example (prices, battery size, loads) are placeholders for planning. The hardware team will confirm real values once the bench is built.

Outputs: what you decide every time step

You reply with set-points for this step. The grid takes up whatever is left over, so you don't set it directly when it is available.

Field	Unit	Meaning
`p_batt_w`	W	Battery power: positive = charge, negative = discharge
`shed`	list of houses	Houses whose non-critical loads switch off this step
`curtail_pv_w`	W	Solar to throw away (only when islanded and the battery is full)
`trades`	list	Who sold energy to whom this step: `{"seller": "H1", "buyer": "H3", "wh": 1.4}`
`p2p_price`	₹/kWh	Price for neighbour-to-neighbour trades this step

Every step must satisfy one equation: **energy in equals energy out**. With P_grid positive for import:

```
\sum_h \mathrm{PV}_h - P_{\mathrm{curt}} + P_{\mathrm{grid}} +
P_{\mathrm{dis}} - P_{\mathrm{ch}} = \sum_h L^{\mathrm{crit}}_h + \sum_h s_h
\, L^{\mathrm{flex}}_h
```

Here s_h is 1 if house h's flexible load is served and 0 if shed. When the grid is down, P_grid = 0, so the battery and shedding must close the gap on their own.

The battery's charge then moves by:

```
\mathrm{SOC}_{t+1} = \mathrm{SOC}_t + \frac{\left(\eta_c P_{\mathrm{ch}} -
P_{\mathrm{dis}} / \eta_d\right)\, \Delta t}{E_{\mathrm{cap}}}
```

Charge and discharge efficiencies are each about the square root of the round-trip efficiency: $\sqrt{0.9} \approx 0.95$. The losses are why arbitrage only pays when the price gap beats them.

Objectives and hard constraints

The goals are ranked, not blended: a higher one is never traded away for a lower one.

1. **Critical loads are always served.** This is a hard constraint, never a penalty.
2. **Safety limits are respected**: SOC between `soc_min` and `soc_max`, battery power within its limits, no export while islanded.
3. **Outage reserve**: keep enough charge to run all critical loads for a set time (e.g. 4 hours) in case the grid fails. A rule-based baseline usually forgets this.
4. **Lowest community cost**: buy cheap, sell dear, use local solar before grid power.
5. **Fair sharing**: neighbour-to-neighbour solar trades before grid exports, and prepaid rules honoured (a house at zero credit loses its flexible load, keeps its critical lifeline).

Goal 4 written as a cost to minimise over a day of steps:

```
\min \sum_t \Big[ c^{\mathrm{buy}}_t P^{\mathrm{imp}}_t - c^{\mathrm{sell}}_t
P^{\mathrm{exp}}_t + \lambda_{\mathrm{shed}} \sum_h (1 - s_{h,t})
L^{\mathrm{flex}}_{h,t} + \lambda_{\mathrm{deg}} (P^{\mathrm{ch}}_t +
P^{\mathrm{dis}}_t) \Big] \Delta t
```

- The **shedding penalty** λ_shed is large, so the optimiser only sheds when it must.
- The **degradation penalty** λ_deg is small and stops the battery cycling for a few paise of profit.
- Value the energy left in the battery at the end of the horizon, or the optimiser will empty it on the last step.

How to build it: four levels, in order

Build level 1 first, even though it isn't ML. It is your baseline for every later comparison and the fallback the hub runs if your manager is down. Each level must beat the one before it on the metrics in the evaluation section, or it doesn't ship.

1. **Rule-based baseline (half a day).** Simple if-then rules:
   - Solar surplus goes to neighbours first, then the battery, then the grid.
   - Charge from the grid only when `price_buy` is in the off-peak band.
   - Discharge in the evening peak, but never below the outage reserve.
   - Grid down: battery covers the deficit; shed flexible loads, lowest-credit house first, if the battery can't.
2. **Optimisation with model predictive control (MPC) (1–2 days). This is the core.** Each step, solve the cost equation above as a linear program over the next 24 h, apply only the first step's decision, then repeat next step with fresh data. Use `cvxpy` or `PuLP` (both free).

- Solve it first with *perfect* forecasts (the real future from the simulator). That gives the best cost achievable, the ceiling for every other method.
- Shedding decisions are yes/no, which makes this a mixed-integer program (MILP). Start by relaxing them to fractions; it is usually fine.

3. **ML forecasting (the ML showcase).** MPC is only as good as its forecasts of solar and demand. Train models to predict the next 24 h of PV and load:
   - Features: hour, day of week, recent values, and irradiance or cloud cover if you add weather.
   - Start with gradient boosting (LightGBM); try an LSTM if there is time.
   - Report how much forecast error costs, measured as the gap to perfect-forecast MPC.
4. **Reinforcement learning (stretch goal).** Wrap the simulator as a `gymnasium` environment and train PPO or SAC (via `stable-baselines3`) to choose battery power directly. It's impressive if it gets near the MPC result, but it is harder to make safe, so keep the hub's hard limits in force.

For neighbour-to-neighbour trade prices, a simple, defensible rule is the midpoint: (price_buy + price_sell) ÷ 2. Both sides beat the grid, so trading is always worth it.

Simulator and data: build these first

You won't have months of real data, so write a small Python simulator. It generates synthetic days, applies your decisions through the two equations above, and scores the result. It doubles as the training environment for forecasting and RL.

Synthetic profiles (15-min steps, 96 per day; generate a year of them):

Signal	How to generate it
Solar `pv_w`	Bell curve from about 06:00 to 18:00, peak at noon; multiply by a random cloud factor (0.3–1.0) that drifts through the day; lower in monsoon months. Houses 1 and 2 get different peak sizes
Critical load	Small and nearly flat, with a little noise
Flexible load	Morning bump (06:00–09:00) and a bigger evening peak (18:00–22:00); weekends shifted later
`price_buy`	Time-of-day tariff (rules below)

Signal	How to generate it
`price_sell`	Flat, well below the buy price (e.g. ₹3/kWh)
`grid_available`	1–3 random outages a day, 30–120 min each, plus one announced outage on some days

For realistic solar shapes, NASA POWER offers free historical irradiance for any Indian city; scale it down to the demo panel size.

Tariff assumptions grounded in the Indian rules. Under the Ministry of Power's 2023 amendment, time-of-day tariffs apply to commercial and industrial users from 1 April 2024 and to most others from 1 April 2025, once smart meters are installed (PIB):

- **Solar hours** (8 hours a day, set by each state) cost at least **20 % less** than normal.
- **Peak hours** cost at least **10 % more** for households, 20 % more for commercial and industrial users.

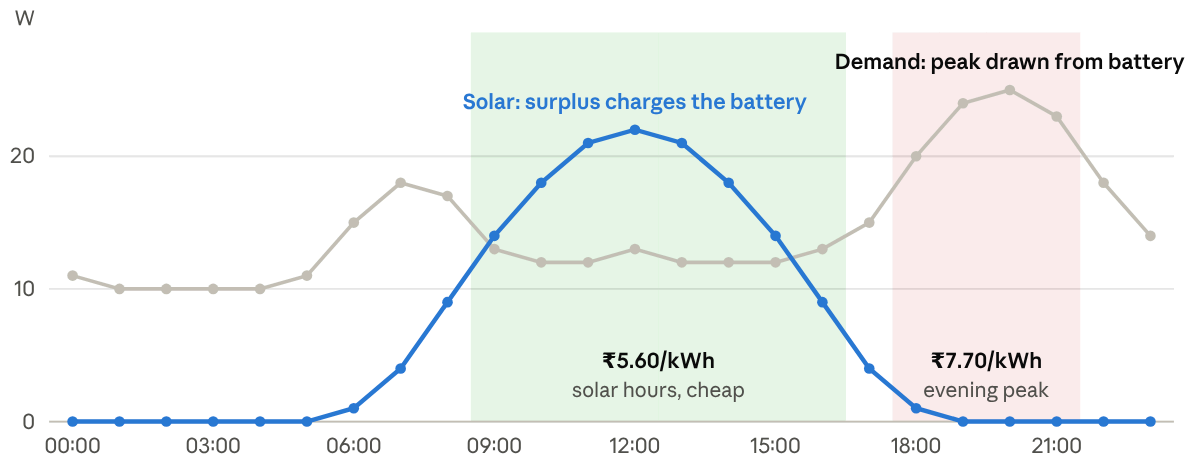
A reasonable simulator tariff: normal ₹7/kWh, solar hours 09:00–17:00 at ₹5.60, evening peak 18:00–22:00 at ₹7.70. These are placeholders; check your state commission's actual rates if a judge asks.

Split by whole days: for example 70 % train, 15 % validation, 15 % test. Never shuffle individual steps, or the forecaster will see the future.

The chart below shows what one simulated sunny weekday looks like and where the arbitrage opportunity sits.

Solar peaks at noon, demand after 18:00: the battery bridges the gap

One simulated sunny weekday, whole community, watts. Shading shows the time-of-day price



illustrative synthetic profile, typed in chat · placeholder tariff based on the 2023 ToD rules

At noon solar exceeds demand by about 9 W. Storing it costs nothing extra, while exporting earns only ₹3/kWh. Spent in the 18:00–22:00 peak, it saves ₹7.70/kWh. That gap, minus about 10 % battery losses, is the arbitrage your manager captures.

Interface with the hardware hub

The simulator and the real hub speak **the same JSON**, so your manager code doesn't change between simulation and the live demo. Only the connection does.

- **Transport**: MQTT over the local Wi-Fi, with a Mosquitto broker on the laptop. MQTT is lightweight and well supported on ESP32 and in Python (`paho-mqtt`).
- **Topics**: the hub publishes the input message on `mgrid/state` ; you publish the output message on `mgrid/setpoint` .
- **Timing**: one message per step. Reply within **5 seconds**.
- **Fallback**: if the hub hears nothing for 2 steps, it switches to its built-in rule-based mode (level 1) and says so on the dashboard. Your manager is never a single point of failure.
- **Clamping**: the hub clips any set-point outside the battery limits and logs that it did. Watch those logs; frequent clipping means your model of the battery is off.
- **Logging**: store every state and set-point pair. That history is your training data once the hardware runs.

Agree the exact field names with the hardware team before either side writes code; the tables above are the starting proposal.

Evaluation, demo and plan

Judge every method on the same held-out test days, against the rule-based baseline and the perfect-forecast MPC ceiling. The headline number for the pitch: **how much of the gap between baseline and ceiling your ML closes.**

Metric	Definition	Better
Community cost	₹ paid to the grid minus ₹ earned from exports, per day	Lower
Critical energy not served	Wh of critical load missed	Must be 0

Metric	Definition	Better
Flexible energy shed	Wh of non-critical load switched off	Lower
Self-sufficiency	Share of demand met by local solar and battery	Higher
Outage survival	Share of outage minutes with all critical loads up	100 %
Battery throughput	Wh charged + discharged per day (a wear proxy)	Lower for the same cost
Forecast error	MAE of solar and load forecasts, W	Lower

Live demo scenarios (pair them with the hardware team):

1. **Sunny afternoon**: House 1 sells surplus to House 3 while the battery charges cheaply; credits move on the dashboard.
2. **Evening peak**: the battery discharges instead of buying expensive grid power. Show the ₹ saved against the baseline.
3. **Grid failure**: islanding, critical loads stay on, flexible loads shed by priority and credit.
4. **Announced outage**: the manager pre-charges the battery beforehand. The baseline doesn't, which is the clearest picture of why the ML matters.

Plan, in order:

- [ ] Agree the JSON fields with the hardware team
- [ ] Simulator + synthetic year of data
- [ ] Level 1 rule-based baseline and its scores
- [ ] Level 2 MPC with perfect forecasts (the ceiling)
- [ ] Level 3 forecasters; MPC with forecasts
- [ ] Results table and charts for the pitch
- [ ] MQTT link to the real hub; live demo dry run
- [ ] Level 4 RL, only if time remains

Reference: what AURA does, and our miniature of it

Smart Grid Analytics, an Indian company, sells **Solvyn**, an AI energy management platform for solar, wind, hybrid and battery plants. **AURA** ("Automated Utility Response & Allocation") is its decision layer: it turns plant data, forecasts and constraints into schedules that the control system executes (Pulse 2.0, AI Impact Commons). That is exactly the split between your energy manager and our hub.

AURA (utility scale)	Our community version
Inputs: live plant telemetry, weather forecasts, India Energy Exchange prices, battery and grid limits	Inputs: hub state message, synthetic or NASA weather, ToD tariff, battery limits
Forecasts demand and price; digital twin of battery and grid	Solar and load forecasters (level 3); the simulator as our digital twin
Optimised dispatch schedules that adapt continuously	Rolling-horizon MPC (level 2)
Treats storage as a "planned economic asset": discharge at high prices, avoid deep cycling and thermal stress	Arbitrage with outage reserve, SOC limits and a degradation penalty
Trading and bidding decisions	Neighbour-to-neighbour trades and prices, prepaid credits
Operator dashboards	Hub web dashboard

AURA serves independent power producers running large plants. Our twist is the same intelligence at **neighbourhood scale, with households trading among themselves and the grid able to disappear**. That is the story to tell judges.

Tools to start with: `pandas`, `cvxpy` or `PuLP` (optimisation), `lightgbm` or `scikit-learn` (forecasting), `gymnasium` + `stable-baselines3` (RL, stretch), `paho-mqtt` (hub link).

Sources: Smart Grid Analytics · Pulse 2.0 on Smart Grid Analytics' funding and AURA · AI Impact Commons: Solvyn AURA · PIB: Time of Day tariff rules